

Manual básico: Diseño e implementación de APIs y microservicios

1. ¿Qué es una API?

Una API (Application Programming Interface) es un mecanismo que permite que diferentes sistemas se comuniquen entre sí mediante un conjunto definido de reglas. En términos prácticos, una API expone funcionalidades de un sistema para que otros sistemas puedan utilizarlas sin necesidad de conocer su implementación interna.

Una API actúa como intermediario entre un cliente (por ejemplo, una aplicación web o móvil) y un servidor, podríamos pensar como un lenguaje entre sistemas que envía solicitudes (requests) y recibe respuestas (responses), generalmente en formato JSON.

La aplicación que envía la solicitud (pensemos que nuestra aplicación necesita información de otra aplicación) se llama cliente, y la que envía la respuesta (quien responde con información) se llama servidor.

Para que podamos ver el clima en nuestro celular, una aplicación instalada en nuestro celular manda una consulta sobre el clima a la base de datos meteorológicos del instituto que es el servidor, el servidor que tiene los datos sobre el clima le va a responder a la aplicación que tenemos en el celular con los datos del clima.

Las API pueden funcionar de cuatro maneras diferentes, según el momento y el motivo de su creación.

API de SOAP

Estas API utilizan el protocolo simple de acceso a objetos. El cliente y el servidor intercambian mensajes mediante XML.

API de RPC

Estas API conocidas como llamadas a procedimientos remotos. El cliente completa una función (o procedimiento) en el servidor, y el servidor devuelve el resultado al cliente.

API de WebSocket

Utiliza objetos JSON para transmitir datos. La API de WebSocket admite la comunicación bidireccional entre las aplicaciones cliente y el servidor.

El servidor puede enviar mensajes de devolución de llamada a los clientes conectados, por lo que es más eficiente que la API de REST.

API de REST

Estas son las API más populares y flexibles que se encuentran en la web actualmente.

El cliente envía las solicitudes al servidor como datos. El servidor utiliza esta entrada del cliente para iniciar funciones internas y devuelve los datos de salida al cliente.

¿Qué son las API de REST?

REST significa *“transferencia de estado representacional”*.

REST define un conjunto de funciones como GET, PUT, DELETE, etc. que los clientes pueden utilizar para acceder a los datos del servidor. Los clientes y los servidores intercambian datos mediante HTTP.

La principal característica de la API de REST es que no tiene estado. La ausencia de estado significa que los servidores no guardan los datos del cliente entre las solicitudes.

Las solicitudes de los clientes al servidor son similares a las URL que se escriben en el navegador para visitar un sitio web. La respuesta del servidor son datos simples, sin la típica representación gráfica de una página web.

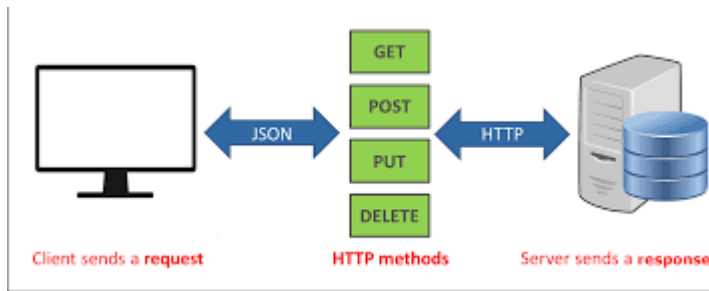
2. Arquitectura general de una API

Una API moderna sigue una arquitectura cliente-servidor basada en HTTP. Sus componentes principales son:

- Cliente: quien realiza la solicitud (frontend, script, otra API).
- Servidor: quien procesa la solicitud.
- Endpoint: URL específica que expone una funcionalidad.
- Método HTTP: define la acción (GET, POST, PUT, DELETE).
- Payload: datos enviados en la solicitud.
- Response: datos devueltos por el servidor.

Flujo básico:

Cliente → Request HTTP → API → Procesamiento → Response → Cliente



3. Tipos de APIs

Las APIs pueden clasificarse según su uso:

- APIs REST: basadas en HTTP, simples y ampliamente utilizadas.
- APIs GraphQL: permiten consultas más flexibles.
- APIs internas: utilizadas dentro de una organización.
- APIs públicas: accesibles por terceros.

En sistemas modernos, REST es el estándar más común para microservicios.

¿Qué es un punto de conexión de la API (Endpoint)?

Los puntos de conexión de las API son los últimos puntos de contacto del sistema de comunicación de las API. Se trata de las URL de servidores, servicios y otras ubicaciones digitales específicas desde las que se envía y recibe información entre sistemas. Los puntos de conexión de las API son fundamentales para las empresas por dos motivos principales:

- Seguridad Los puntos de conexión de las API hacen que el sistema sea vulnerable a los ataques. La supervisión de las API es crucial para evitar su uso indebido.
- Rendimiento Los puntos de conexión de las API, especialmente los de alto tráfico, pueden provocar cuellos de botella y afectar al rendimiento del sistema.

4. Diseño de un sistema mínimo basado en API

Un sistema mínimo debe incluir:

Endpoint principal

Define una ruta clara: POST /generar-noticia

Entrada estructurada

```
{ "keywords": ["inflación argentina", "economía"] }
```

Lógica interna

El backend debe:

Recibir input → Procesar datos → Construir contexto → Llamar a un modelo o lógica de negocio → Generar respuesta estructurada

Salida estructurada

```
{ "texto": "...", "resumen": "..." }
```

Arquitectura de un microservicio

Un microservicio es una unidad independiente que realiza una tarea específica y se comunica mediante APIs.

Estructura básica:

- API layer (FastAPI/Flask)
- Lógica de negocio
- Integraciones externas (APIs, scraping, LLM)
- Validación (Pydantic)
- Logging

Flujo:

Request → API → Lógica → Servicios externos → Procesamiento → Response

6. Implementación mínima en Python

Entre las opciones está usar FastAPI o levantar el servicio por Flask

Vamos a ver un ejemplo con flask por la versatilidad que tiene para ampliar el proyecto.

[Ejemplo_API_LOCAL](#)

